

Lecture 6: Classes

COMP2221: Functional programming

Maximilien Gadouleau

`m.r.gadouleau@durham.ac.uk`

Polymorphism

Polymorphism

- Recall, Haskell is *strictly typed*.
- What does this mean for (say) `length`?

Different types?

```
length [True, False, True] -- :: [Bool] -> Int ?  
length [1, 2, 3]           -- :: [Int] -> Int ?
```

These functions must have *different* types, no?

Polymorphism

- Recall, Haskell is *strictly typed*.
- What does this mean for (say) `length`?

Different types?

```
length [True, False, True] -- :: [Bool] -> Int ?  
length [1, 2, 3]           -- :: [Int] -> Int ?
```

These functions must have *different* types, no?

Polymorphic types

```
Prelude> :type length  
length :: [a] -> Int
```

“`length` eats a list of values of any type `a` and returns an `Int`”

`a` is called a *type variable*.

This is called *parametric polymorphism*.

Contrast with OO languages: definitions

Definition (Parametric polymorphism)

Write a *single* implementation of a function that applies generically *and identically* to values of any type.

Definition (“ad-hoc” polymorphism)

Write *multiple* implementations of a function, one for each type you wish to support.

Definition (Subtype polymorphism)

Relate datatypes by some “substitutability”. Write a function for a supertype instance. Now all subtypes can use it.

“Duck typing” or “Liskov substitution principle”.

Contrast with OO languages: examples

Subtype polymorphism

```
class Foo(object):
    def length(self, ...):
        pass
class Bar(Foo):
    pass
a = Foo().length()
# Every Bar is-a Foo, so we can
# call the length method.
b = Bar().length()
```

Ad-hoc polymorphism

```
class Foo(object):
    pass
class Bar(object):
    pass
def length(obj):
    if isinstance(obj, Foo):
        ...
    elif isinstance(obj, Bar):
        ...
# length knows how to handle things
# of type Foo and type Bar
a = length(Foo())
b = length(Bar())
```

Parametric polymorphism

```
-- length doesn't care what type the entries
-- in the list are
length :: [a] -> Int
length [] = 0
length (_,xs) = 1 + length xs
```

Contrast with OO languages

- Parametric polymorphism also called *generic programming*
- Introduced in ML in 1975.
- Has been adopted by a number of languages, including traditional OO ones.
- For example, Java or C# have “generics” for this purpose

```
// Implementation of HashSet is generic  
// Specialised on instantiation  
Set<int> intset = new HashSet<int>();  
Set<Object> objset = new HashSet<Object>();
```

- C++ templates also allow for similar style of programming

Constraining polymorphic functions

- Some polymorphic functions only apply to types that satisfy certain constraints
- For example `(+)` works on all types `a`, *as long as* that type is a number type.

Example

`(+) :: Num a => a -> a -> a`

“For any type `a` that is an *instance* of the *class* `Num` of numeric types, `(+)` has type `a -> a -> a`”

- This constraint is called a *class constraint*
 - An expression or type with one or more such constraints is called *overloaded*.
- ⇒ `Num a => a -> a -> a` is an *overloaded type* and `(+)` is an *overloaded function*.

WARNING!

The *words* class and instance are the same as in object-oriented programming languages, but their *meaning* is very different.

Definition (Class)

A collection of *types* that support certain, specified, overloaded operations called *methods*.

Definition (Instance)

A concrete type that belongs to a *class* and provides implementations of the required methods.

Analogous constructs in other languages

- Compare: type “a collection of related values”
- This is *not* like subclassing and inheritance in Java/C++
- If you write flat interfaces with ‘abc.abstractmethod’ in Python.
- Rust traits give you something close
- Close to a combination of Java *interfaces* and *generics*
- C++ “concepts” (in C++20) are also very similar.

Here are some basic type classes used in Haskell

- **Eq**: types with equality
- **Ord**: ordered types (subclass of **Eq**)
- **Read**: types one can read from the IO
- **Show**: types that can be displayed
- **Num**: numerical types, that allow addition and multiplication

Defining classes I

- Let us say we want to encapsulate some new property of types

Foo-ness

- We define the interface the type should support

```
class Foo a where  
  isfoo :: a -> Bool
```

- Now we say how types implement this

```
instance Foo Int where  
  isfoo _ = False  
  
instance Foo Char where  
  isfoo c = c `elem` ['a'..'c']
```

- Can add new interfaces to old types, and new types to old interfaces.
- Contrast Java, where if I implement a new interface it is very difficult to make existing classes implement it.

Defining classes II

- Classes (interfaces) can provide default implementation.
- Example, the `Eq` class representing equality requires both `(==)` and `(/=)`.
- Since $a == b \Leftrightarrow \text{not } (a /= b)$, we can provide *default* implementations and only require that an instance implements one.

```
class Eq a where
  (==) :: a -> a -> Bool
  x == y = not (x /= y)
  (/=) :: a -> a -> Bool
  x /= y = not (x == y)
```

-- instance for MyType only needs to provide one of (==) or (/=).

```
instance Eq MyType where
  x == y = ...
```